

pytest — Complete Beginner Textbook

What it is • How to use it • How it works • Examples

Source preserved verbatim from the preceding conversation. Formatting is applied to improve textbook readability.

Absolutely. Since you already know **basic Python**, the easiest way to understand `pytest` is to think of it as:

```
> A tool that automatically runs your Python code with test cases and tells you whether your code behaves correctly.
```

It is one of the most commonly used testing frameworks in Python, especially for backend development with things like **FastAPI**.

1. Why do we need pytest?

Suppose you write this function:

```
def add(a, b):  
    return a + b
```

You can manually test it:

```
print(add(2, 3))  
print(add(10, 20))  
print(add(-1, 5))
```

You look at the output:

```
5  
30  
4
```

This works for a tiny function.

But imagine you have a project with:

```
50 Python files  
200 functions  
20 API endpoints  
database operations  
authentication  
business logic
```

Manually checking everything after every change becomes painful.

That's where **automated testing** comes in.

Instead of:

```
print(add(2, 3))
```

you write a test:

```
def test_add():  
    assert add(2, 3) == 5
```

Then `pytest` runs the test for you.

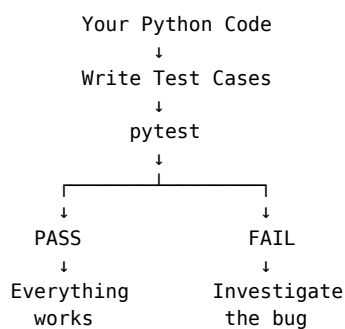
2. What is pytest?

`pytest` is a **Python testing framework**.

It allows you to:

- Write test cases
- Automatically discover tests
- Run many tests at once
- Compare expected vs actual results
- Identify exactly which tests failed
- Test exceptions
- Test APIs
- Test classes and objects
- Test asynchronous code
- Use reusable test fixtures
- Generate test reports

The basic idea is:



3. Installing pytest

You install it using `pip`:

```
pip install pytest
```

Verify:

```
pytest --version
```

You should see something similar to:

```
pytest 8.x.x
```

4. Your first pytest test

Let's create a small project.

```
my_project/
├──
├── calculator.py
└── test_calculator.py
```

`calculator.py`

```
def add(a, b):
    return a + b
```

Now create:

`test\_calculator.py`

```
from calculator import add

def test_add():
    assert add(2, 3) == 5
```

Run:

```
pytest
```

You might see:

```
===== test session starts =====
collected 1 item

test_calculator.py .                                [100%]

===== 1 passed in 0.02s =====
```

The important part:

```
1 passed
```

Your test succeeded.

5. The most important concept: `assert`

If you're learning pytest, ****understand `assert` first****.

Python already has an `assert` statement.

For example:

```
x = 10
assert x == 10
```

This succeeds.

But:

```
x = 10
assert x == 20
```

fails.

You can think of:

```
assert actual == expected
```

as:

```
> "Python, verify that this condition is true."
```

With pytest:

```
def test_add():
    assert add(2, 3) == 5
```

pytest executes:

```
add(2, 3)
```

which gives:

```
5
```

Then checks:

```
5 == 5
```

Therefore:

```
PASS
```

```
---
```

6. Testing multiple cases

You don't need just one test.

```
from calculator import add

def test_add_positive_numbers():
    assert add(2, 3) == 5

def test_add_negative_numbers():
    assert add(-2, -3) == -5

def test_add_positive_and_negative():
    assert add(10, -3) == 7
```

Run:

```
pytest
```

Result:

```
3 passed
```

Now imagine someone changes your implementation:

```
def add(a, b):
    return a - b
```

Your tests will catch it.

```
FAILED test_calculator.py::test_add_positive_numbers
FAILED test_calculator.py::test_add_negative_numbers
FAILED test_calculator.py::test_add_positive_and_negative
```

That's the value of automated testing.

7. How pytest finds your tests

This is something beginners often wonder about.

You don't normally tell pytest:

```
Run test_calculator.py
```

You simply run:

```
pytest
```

pytest searches for test files.

Common naming conventions include:

```
test_*.py
*_test.py
```

For example:

```
test_calculator.py
test_user.py
test_api.py
```

Inside those files, pytest looks for functions whose names start with:

```
test_
```

For example:

```
def test_add():
    ...

def test_login():
    ...

def test_create_user():
    ...
```

So:

```
pytest
↓
find test files
↓
find test functions
↓
execute tests
↓
evaluate assertions
↓
report results
```

8. A test is just Python code

This is very important.

A pytest test isn't some magical new programming language.

This:

```
def test_add():
    assert add(2, 3) == 5
```

is simply a Python function containing an assertion.

pytest provides the machinery to:

- discover it

- execute it
- capture failures
- show useful debugging information
- organize many tests

9. Testing different types of values

You can test anything you normally compare in Python.

Numbers

```
def test_number():
    assert 10 + 5 == 15
```

Strings

```
def test_string():
    name = "Google"

    assert name == "Google"
```

Lists

```
def test_list():
    numbers = [1, 2, 3]

    assert numbers == [1, 2, 3]
```

Dictionaries

```
def test_dictionary():
    user = {
        "name": "John",
        "age": 30
    }

    assert user["name"] == "John"
```

Boolean

```
def test_boolean():
    result = True

    assert result is True
```

10. Testing a real Python function

Suppose you have:

```
def is_even(number):  
    return number % 2 == 0
```

Your test:

```
def test_is_even():  
    assert is_even(2) is True  
    assert is_even(4) is True  
    assert is_even(10) is True
```

And:

```
def test_is_odd():  
    assert is_even(3) is False  
    assert is_even(7) is False
```

Notice something interesting.

One test can have multiple assertions:

```
def test_is_even():  
    assert is_even(2) is True  
    assert is_even(4) is True  
    assert is_even(10) is True
```

But generally, keeping tests focused makes failures easier to understand.

11. Testing exceptions

Suppose you have:

```
def divide(a, b):  
    return a / b
```

What happens here?

```
divide(10, 0)
```

Python raises:

```
ZeroDivisionError
```

We can test that this is intentional.

pytest provides:

```
pytest.raises()
```

Example:

```
import pytest  
  
def test_divide_by_zero():
```

```
with pytest.raises(ZeroDivisionError):
    divide(10, 0)
```

This means:

```
> "I expect this piece of code to raise `ZeroDivisionError`."
```

If it does:

PASS

If it doesn't:

FAIL

12. Testing your own custom exceptions

Suppose:

```
class InvalidAgeError(Exception):
    pass

def check_age(age):
    if age < 18:
        raise InvalidAgeError("Must be 18 or older")

    return True
```

Test:

```
import pytest

def test_invalid_age():
    with pytest.raises(InvalidAgeError):
        check_age(15)
```

You can also verify the error message:

```
def test_invalid_age_message():
    with pytest.raises(InvalidAgeError, match="18 or older"):
        check_age(15)
```

This is extremely useful in real applications.

13. Testing a class

Suppose you have:

```
class Calculator:

    def add(self, a, b):
        return a + b

    def multiply(self, a, b):
        return a * b
```

Tests:

```
from calculator import Calculator

def test_calculator_add():
    calculator = Calculator()

    assert calculator.add(2, 3) == 5

def test_calculator_multiply():
    calculator = Calculator()

    assert calculator.multiply(2, 3) == 6
```

pytest doesn't care whether your code contains:

- functions
- classes
- objects
- modules

It simply executes your tests.

14. Fixtures — one of the most important pytest concepts

As projects become bigger, you'll notice that tests often need some setup.

Imagine 10 tests all need a user:

```
user = {
    "name": "John",
    "email": "john@example.com"
}
```

You don't want to repeat this everywhere.

pytest provides **fixtures**.

Example:

```
import pytest
```

```
@pytest.fixture
def user():
    return {
        "name": "John",
        "email": "john@example.com"
    }
```

Now use it:

```
def test_user_name(user):
    assert user["name"] == "John"

def test_user_email(user):
    assert user["email"] == "john@example.com"
```

pytest sees:

```
def test_user_name(user):
```

and recognizes:

```
> "This test needs the `user` fixture."
```

It executes:

```
user()
```

and passes the returned value into the test.

Conceptually:

```
@pytest.fixture
  ↓
user()
  ↓
creates test data
  ↓
user
  ↓
┌───┴───┐
↓         ↓
test_user_name test_user_email
```

Fixtures become ****very important when you start testing FastAPI applications****.

15. Why fixtures are powerful

Imagine testing an application that needs:

```
Database
User
Authentication token
API client
Configuration
Temporary files
```

Instead of manually setting them up in every test, fixtures can provide them.

For example:

```
@pytest.fixture
def user():
    return create_test_user()
```

Then:

```
def test_user_login(user):
    ...
```

This keeps your tests clean.

16. Parametrization — testing many inputs

Suppose:

```
def add(a, b):
    return a + b
```

You could write:

```
def test_add_1():
    assert add(1, 2) == 3

def test_add_2():
    assert add(10, 20) == 30

def test_add_3():
    assert add(-1, 1) == 0
```

But pytest gives you a cleaner way:

```
import pytest

@pytest.mark.parametrize(
    "a,b,expected",
    [
        (1, 2, 3),
        (10, 20, 30),
        (-1, 1, 0),
        (100, 200, 300),
    ]
)
def test_add(a, b, expected):
    assert add(a, b) == expected
```

pytest effectively runs:

```
test_add(1, 2, 3)
test_add(10, 20, 30)
```

```
test_add(-1, 1, 0)
test_add(100, 200, 300)
```

So you get:

```
4 passed
```

This is extremely useful for algorithm testing.

17. Example: testing a DSA function

Suppose you're implementing Two Sum:

```
def two_sum(numbers, target):
    seen = {}

    for i, number in enumerate(numbers):
        complement = target - number

        if complement in seen:
            return [seen[complement], i]

    seen[number] = i

    return []
```

You can test it with:

```
def test_two_sum():
    assert two_sum([2, 7, 11, 15], 9) == [0, 1]
```

More cases:

```
import pytest

@pytest.mark.parametrize(
    "numbers,target,expected",
    [
        ([2, 7, 11, 15], 9, [0, 1]),
        ([3, 2, 4], 6, [1, 2]),
        ([3, 3], 6, [0, 1]),
    ]
)
def test_two_sum(numbers, target, expected):
    assert two_sum(numbers, target) == expected
```

This is a great habit for DSA.

Instead of only asking:

```
> "Does my algorithm work for this example?"
```

you ask:

```
> "Does my algorithm work across multiple test cases?"
```

18. pytest for FastAPI

This is particularly relevant to your current backend learning.

Suppose you eventually create:

```
FastAPI application
  ↓
GET /users
POST /users
GET /users/{id}
DELETE /users/{id}
```

You can write tests for your APIs.

For example:

```
from fastapi.testclient import TestClient

from main import app

client = TestClient(app)

def test_root():
    response = client.get("/")

    assert response.status_code == 200
```

You can test the response:

```
def test_get_user():
    response = client.get("/users/1")

    assert response.status_code == 200
    assert response.json()["id"] == 1
```

So now you're testing not just Python functions but your **API behavior**.

19. What happens when a test fails?

Suppose:

```
def add(a, b):
    return a - b
```

Test:

```
def test_add():
```

```
assert add(2, 3) == 5
```

Run:

```
pytest
```

pytest might report:

```
FAILED test_calculator.py::test_add
E       assert -1 == 5
E       + where -1 = add(2, 3)
```

This is much better than:

```
print(add(2, 3))
```

because pytest tells you:

```
Expected: 5
Actual:   -1
```

and points you toward the failing test.

20. Running specific tests

You don't always have to run everything.

Run all tests:

```
pytest
```

Run one file:

```
pytest test_calculator.py
```

Run one specific test:

```
pytest test_calculator.py::test_add
```

Run tests matching a keyword:

```
pytest -k "add"
```

For example:

```
test_add
test_add_negative
test_add_large_numbers
```

could all be selected.

21. Useful pytest options

Verbose output

```
pytest -v
```

Instead of:

```
...
```

you get something like:

```
test_calculator.py::test_add PASSED
test_calculator.py::test_multiply PASSED
```

Stop at first failure

```
pytest -x
```

Useful when debugging.

Show print statements

Normally pytest captures output.

You can use:

```
pytest -s
```

Example:

```
def test_example():
    print("Hello from test")
    assert True
```

Run:

```
pytest -s
```

and you'll see the output.

22. A realistic project structure

As your Python/FastAPI projects become larger, you'll commonly see something like:

```
my_project/
├── app/
│   ├── main.py
│   ├── models.py
│   ├── services.py
│   └── utils.py
├── tests/
│   ├── test_main.py
│   ├── test_services.py
│   └── test_utils.py
├── requirements.txt
└── pyproject.toml
```

Then:

```
pytest
```

can run the entire test suite.

23. Unit tests vs integration tests

You'll encounter these terms frequently in software engineering interviews.

Unit test

Tests one small piece of functionality.

Example:

```
def test_add():
    assert add(2, 3) == 5
```

Conceptually:

```
add()
  ↓
TEST IT
  ↓
PASS
```

Integration test

Tests multiple components working together.

For example:

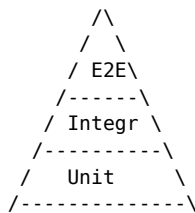
```
API
↓
Service
↓
Database
```

An integration test might check:

```
POST /users
↓
FastAPI
↓
User Service
↓
Database
↓
User created
```

24. The testing pyramid

A useful concept to remember:



Usually:

****Many unit tests****

```
Functions
Business logic
Utilities
Algorithms
```

****Some integration tests****

```
API + database
Service + repository
```

****Few end-to-end tests****

```
Complete user workflow
```

Why?

Unit tests are generally:

- fast
- easy to debug

- isolated

End-to-end tests are generally:

- slower
- more complicated
- more expensive to maintain

25. How pytest actually works

Let's understand the execution flow.

You run:

```
pytest
```

Step 1 — pytest starts

```
pytest
↓
Initialize testing framework
```

Step 2 — Test discovery

pytest searches for files such as:

```
test_*.py
*_test.py
```

Step 3 — Find tests

Inside those files it finds things like:

```
def test_add():
```

Step 4 — Execute

pytest calls:

```
test_add()
```

Step 5 — Assertions execute

```
assert add(2, 3) == 5
```

Step 6 — pytest records the result

```
PASS
```

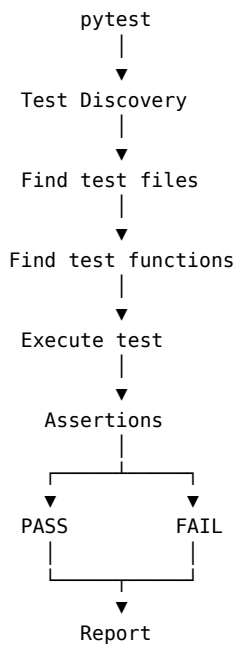
or:

FAIL

Step 7 — Report

10 passed, 2 failed

So the complete picture is:



26. pytest vs `unittest`

Python also has a built-in testing framework called `unittest`.

`unittest`:

```
import unittest

class TestCalculator(unittest.TestCase):

    def test_add(self):
        self.assertEqual(add(2, 3), 5)
```

pytest:

```
def test_add():
    assert add(2, 3) == 5
```

pytest is popular because its syntax is generally simpler and it provides a powerful ecosystem around fixtures, parametrization, plugins, and test discovery.

For your current learning path, ****pytest is a very good framework to learn first****.

27. What you should learn in pytest

Don't try to memorize everything.

For your Python → FastAPI → AI/ML engineering path, learn these in roughly this order:

Level 1 — Essential

```
pytest
test_*.py
test_ functions
assert
pytest command
pytest -v
```

Level 2 — Very important

```
pytest.raises()
fixtures
@pytest.fixture
@pytest.mark.parametrize
```

Level 3 — Backend testing

```
FastAPI TestClient
API testing
status codes
JSON response testing
request body testing
headers
authentication
database testing
```

Level 4 — Professional testing

```
mocking
monkeypatch
conftest.py
fixtures with scopes
async tests
coverage
CI/CD
```

28. A small complete example

Let's put everything together.

`calculator.py`

```
def add(a, b):
```

```

        return a + b

def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")

    return a / b

```

`test\_calculator.py`

```

import pytest

from calculator import add, divide

def test_add():
    assert add(2, 3) == 5

@pytest.mark.parametrize(
    "a,b,expected",
    [
        (10, 2, 5),
        (20, 4, 5),
        (9, 3, 3),
    ]
)
def test_divide(a, b, expected):
    assert divide(a, b) == expected

def test_divide_by_zero():
    with pytest.raises(ValueError):
        divide(10, 0)

```

Run:

```
pytest -v
```

Conceptually:

```

test_add
  ↓
2 + 3 == 5
  ↓
PASS

test_divide [10,2]
  ↓
10 / 2 == 5
  ↓
PASS

test_divide [20,4]
  ↓
20 / 4 == 5
  ↓
PASS

test_divide [9,3]
  ↓
9 / 3 == 3
  ↓
PASS

test_divide_by_zero

```

```
↓
ValueError expected
↓
ValueError raised
↓
PASS
```

Result:

```
5 passed
```

29. The mental model I want you to remember

Don't think:

```
> "pytest is a complicated testing library."
```

Think:

```
> """I write Python code that describes what my program should do, and pytest automatically
runs those checks."""
```

For example:

```
def calculate_discount(price):
    return price * 0.9
```

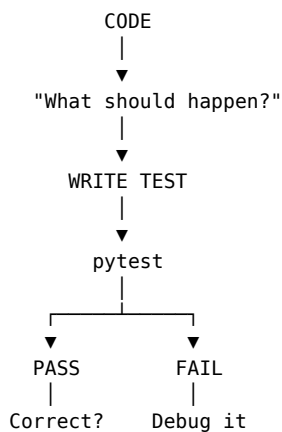
Your requirement is:

```
> ₹100 should become ₹90.
```

So you write:

```
def test_calculate_discount():
    assert calculate_discount(100) == 90
```

That's essentially the heart of pytest.



And this becomes particularly important for your **FastAPI project**: every endpoint you build should eventually have tests proving that it returns the correct status code, response data, and error behavior.